

Bagging and Random Forests

Date: March 5, 2026

Recall decision trees from two lectures ago. They recursively split the feature space to create a flowchart-like model where each internal node represents a decision and each leaf contains a prediction. They're interpretable and handle different data types well, but they have a big problem: when grown to full depth, they overfit. Full-depth decision trees can perfectly memorize the training data instead of learning generalizable patterns. In the previous lecture on bias and variance, we saw why: decision trees have low bias but high variance, and they are too sensitive to the specific examples they see during training.

Random forests solve this problem elegantly: they combine multiple decision trees to reduce variance while keeping the low bias of decision trees. The idea is simple - train many trees on different random subsets of your data, then average their predictions. Unlike kernel methods, neural networks, or regularized models that need careful tuning, random forests typically work well out-of-the-box across many problems.

Today we'll see how random forests work, why they're effective, and what makes them one of the best general-purpose algorithms in machine learning.

1 Bagging

Bagging is a portmanteau of **bootstrap aggregating**. The old terminology stems from the colloquial phrase “pulling oneself up by their bootstraps.” Originally designed as a way to combat not having enough data, bagging has become a popular general tool to address so-called “overfitting” to training data. As we saw in the previous lecture on bias-variance decomposition, overfitting occurs when a model has high variance. Bagging directly targets and reduces this variance component of the generalization error.

Regardless of the above discussion, bagging is actually extremely simple. As usual, we start with a supervised learning dataset:

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

What we do is we draw T new datasets $\mathcal{D}_1, \dots, \mathcal{D}_T$, also of size n with *replacement* from this dataset. In other words, when we make $\mathcal{D}_1$, we choose n data points from the original dataset $\mathcal{D}$. Each data point we choose is drawn at random from $\mathcal{D}$, and we can choose the same data point multiple times.

Example. Suppose we have a supervised learning dataset consisting of 5 training examples:

$$\mathcal{D} : \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5)\}$$

The first dataset we create for bagging, $\mathcal{D}_1$, will also contain 5 training examples. Each training example will be one of the above five picked without replacement. To achieve this, I roll a dice

with sides 1, 2, 3, 4, 5 five times and take the data point I get each time. Let's say the rolls came up 2, 5, 2, 4, 4. Then:

$$\mathcal{D}_1 : \{(\textcolor{red}{x}_2, \textcolor{red}{y}_2), (\textcolor{violet}{x}_5, \textcolor{violet}{y}_5), (\textcolor{red}{x}_2, \textcolor{red}{y}_2), (\textcolor{teal}{x}_4, \textcolor{teal}{y}_4), (\textcolor{teal}{x}_4, \textcolor{teal}{y}_4)\}$$

For $\mathcal{D}_2$, I roll the dice 5 more times. Let's say the rolls come up 1, 3, 2, 3, 5. Then:

$$\mathcal{D}_2 : \{(\textcolor{brown}{x}_1, \textcolor{brown}{y}_1), (\textcolor{blue}{x}_3, \textcolor{blue}{y}_3), (\textcolor{red}{x}_2, \textcolor{red}{y}_2), (\textcolor{blue}{x}_3, \textcolor{blue}{y}_3), (\textcolor{violet}{x}_5, \textcolor{violet}{y}_5)\}$$

Repeating this procedure results in T datasets gives us $\mathcal{D}_1, \dots, \mathcal{D}_T$.

Bagged models. Recall that a machine learning algorithm $\mathcal{A}$ takes as input a dataset $\mathcal{D}$ and returns a model $h_{\mathcal{D}}$ that has been trained on that dataset. To augment $\mathcal{A}$ with bagging, we simply run $\mathcal{A}$ on each of the bootstrapped datasets ("bags") and average the result:

$$\begin{aligned} h_{\mathcal{D}_1}(x) &= \mathcal{A}(\mathcal{D}_1) \\ h_{\mathcal{D}_2}(x) &= \mathcal{A}(\mathcal{D}_2) \\ &\vdots \\ h_{\mathcal{D}_T}(x) &= \mathcal{A}(\mathcal{D}_T). \end{aligned}$$

In other words, we train our machine learning algorithm on each bag $\mathcal{D}_t$. We then combine the predictions of each model to get a final, bagged model. In the regression setting, we might average the predictions:

$$H(x) = \frac{1}{T} \sum_{t=1}^T h_{\mathcal{D}_t}(x)$$

In the classification setting, we might take the majority vote:

$$H(x) = \text{mode}\{h_{\mathcal{D}_1}(x), \dots, h_{\mathcal{D}_T}(x)\}$$

1.1 How diverse are the models $h_{\mathcal{D}_t}$?

A natural question to ask is how much overlap we expect to exist in the models we get while doing bagging. Consider a single training example (x_i, y_i) in the original dataset $\mathcal{D}$. What fraction of the datasets $\mathcal{D}_1, \dots, \mathcal{D}_T$ will actually contain this specific training example? Since the datasets are drawn independently, this is equivalent to asking what the probability of including (x_i, y_i) in a dataset is.

The probability that we choose (x_i, y_i) to be the first point in $\mathcal{D}_t$ is $\frac{1}{n}$. Since we are sampling *with replacement*, the probability we choose it in the second position is also $\frac{1}{n}$. In general, for (x_i, y_i) to not appear in $\mathcal{D}_t$ at all, we need to have not chosen it n times. The probability of this is:

$$\left(1 - \frac{1}{n}\right)^n$$

where $1 - 1/n$ is the probability of not choosing the point, and we repeat that n times. For extremely large datasets, we can consider the limit of this expression as $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e} \approx 0.3678...$$

This is the probability that a dataset $\mathcal{D}_t$ does not contain a specific training example. Therefore, we expect any particular training example to not appear in about 37% of the datasets we created for bagging.

2 Random Forests

Random forests are bagged decision trees usually with a small twist (we will discuss shortly). We take our dataset $\mathcal{D}$ and create T datasets $\mathcal{D}_1, \dots, \mathcal{D}_T$ using the procedure above. Then, we train full depth decision trees $h_{\mathcal{D}_1}, \dots, h_{\mathcal{D}_T}$ on these new datasets. Finally, we combine these full depth decision trees as discussed above, that is, on the regression setting, we average the predictions, and in classification settings, we take the majority. Recall from the previous lecture that full depth decision trees have high variance but low bias - they fit the training data perfectly but often don't generalize well. By bagging them, we retain their low bias while significantly reducing their variance.

2.1 The twist: subsampling features

In general, we want the trees in our forest to be as diverse as possible. One way to make the model more diverse is to force the splits to be made with different subsets of features too. When training $h_{\mathcal{D}_t}$, at each node when considering a split, we consider all possible splits from among only $k < d$ of the features rather than all possible splits from all possible features. We choose these k features randomly, and we choose a different set of k features every time we split in all trees.

This forces the trees to be even more diverse, because the splits are forced to use different features. This idea is in general so good that it's usually used by default in software implementations of random forests: Scikit Learn uses $k = \sqrt{d}$ by default.

So, random forests are:

1. Bagged, full depth decision trees.
2. For each decision tree, and at every split, choose splits from among only $k < d$ of the features (commonly: $k = \sqrt{d}$).

3 Advantages of Random Forests

Random forests combine the simplicity of decision trees with powerful ensemble techniques to create one of the most versatile machine learning algorithms available. They handle both classification and regression problems equally well, require minimal preprocessing, and often produce excellent results with default parameters.

Random forests are a *fantastic* machine learning algorithm, with a number of advantages. Even before we get to the advantages of bagging specifically, let's talk about some advantages they inherit from decision trees in general:

Scale insensitivity. A lot of algorithms we've seen so far are really sensitive to the scale of the features in your data. Distance based methods like k -NN and some kernels can be thrown off if some of your features have really big values compared to others (think square footage versus number of bedrooms). Regularization also had a challenge with this. Decision trees don't care! Since they split on the features independently, choosing a split of 3000 square feet is the same regardless of if you normalize that feature value.

Categorical features. Imagine you are predicting housing prices, and one of your features takes values 1, 2, or 3, where 1 means the house is a townhouse, 2 means the house is an condo, and 3 means the house is a single family home. This is a categorical feature, where we happen to encode it as a number but it doesn't necessarily make sense to think of townhouses as more different than single family homes than condos are, even though $|1 - 3| > |2 - 3|$. Decision trees are about the only algorithm we've studied so far that could handle this really naturally: just allow for equality splits (e.g., "is the house a townhouse?")!

3.1 Out-of-bag estimation

Ordinarily, if you want to estimate the performance of your machine learning algorithm, you need to take your training dataset $\mathcal{D}$ and split it up into a training dataset and a separate validation dataset that you don't include during training. While this seems mandatory because your models will always do better on data it saw during training than on new data, it can also be pretty painful if you don't have huge amounts of data.

Random forests give us an alternative approach to estimating generalization error: out of bag error. The basic idea is that, for any training example (x_i, y_i) there is some set of trees in the forest (about 40% of them) that actually weren't trained on that training example. So, for each training example, let's just measure the performance of all the trees that didn't see that training example.

More formally, for classification let's define:

$$H_{-i}(x) = \text{mode}\{h_{\mathcal{D}_t}(x) | (x_i, y_i) \notin \mathcal{D}_t\}.$$

That is, $H_{-i}(x)$ is the prediction of the random forest, but excluding trees that were trained on (x_i, y_i) . The out of bag error is:

$$\text{err}_{\text{oob}} = \frac{1}{n} \sum_{i=1}^n \ell_{0/1}(H_{-i}(x_i), y_i)$$

Stripping away the mathematics, this is the exact same thing as the *training error* of the random forest, but for each training point x_i we only predict using the trees in the random forest that weren't trained on x_i . Since the trees do not use the input x_i , it can also in some sense be thought of the *test error*.

For regression, you'd just define $H_{-i}(x)$ to be the average over the set of trees that do not use training sample i and the corresponding error using mean squared as the loss.

3.2 Uncertainty quantification

When we studied Gaussian processes, one of the killer advantages was the fact that, for any test example, our model gave us a variance estimate that effectively served as an error bar on our prediction. Random forests can achieve something pretty similar, although we don't get a nice simple Gaussian distribution.

Rather than considering only the full forest's prediction, $H(x)$, we can view the individual tree predictions $h_{\mathcal{D}_1}(x), \dots, h_{\mathcal{D}_T}(x)$ as individual estimates of the label of x . We can use this set of "sample predictions" to reason about our confidence in the prediction of x . For example, if the variance

$$\text{Var}[h_{\mathcal{D}_1}(x), \dots, h_{\mathcal{D}_T}(x)]$$

across our predictions is small, that means that all of our trees are making roughly the same prediction for x , which is one notion of being very confident in that prediction. On the other hand, if this variance is large, it means that different trees lead to very different predictions for x , and so our "uncertainty" is large.

3.3 Out of bag pruning

Very briefly, one final thing we can use the out of bag error estimates for is pruning. When we prune a decision tree, we merge two leaf nodes into a single leaf node and delete the split that originally lead to being two leaf nodes. Merging two leaf nodes will always change the training error of a full depth decision tree (otherwise you wouldn't have made that split in the first place), but it might not change the out of bag error: the error that tree makes on data it wasn't trained on. If merging two leaf nodes doesn't change the out of bag error at all, it's usually a strictly good idea to prune.

We won't cover pruning very much because most implementations of decision trees will already do this for you, but we'll go over an example in class.